# Production Considerations

### Scalability

1. Move from single-process in-memory state to shared storage (for example Redis + database) so multiple app instances can serve traffic consistently.
2. Run multiple server instances behind a load balancer, and automatically increasing/decreasing the number of instances based on traffic via load balancing.
3. Although not directly part of the agent script, the ticketing system used to handle escalation requests should implement queues and agent routing.

### Monitoring and observability

1. All messages should be stored in a database as conversation records, and all logs should be linked to both the conversation and the user for full traceability and auditing.
2. Use professional observability tooling for high-traffic environments, such as Grafana, to centralize logging, monitoring, and alerting.

### Security

1. Store secrets in a managed secret vault, never in plain env files on hosts.
2. API: Enforce authentication, rate limiting, input validation, and strict outbound allowlists.
3. Integrate prompt injection prevention defences, including structural prompt isolation, and dual-LLM verification layers. (I would implement this only if we see abuse).

### Reliability

1. Limit the conversation context sent to OpenAI to the latest 100 user-agent messages.
2. The full conversation transcript sent on escalation should be limited to the latest 50 messages.
3. Use canary deployment (or a similar progressive rollout strategy) when releasing update.
4. Implement robust API retry mechanisms.
5. Enforce idempotency and concurrency controls on booking updates to eliminate race conditions and data corruption.
6. Optimize AI token usage by managing string lengths and text truncation
7. Design a multi-provider fallback architecture to automatically route requests to alternative LLM providers in the event of an outage or service disruption.

## Testing

1. Expand automated coverage: unit tests for single functions, conversation simulation tests, monitoring of drop in completion rate, spike in escalations, spike in API/model errors.
2. Validate performance and functionality under peak traffic.
3. Live debugging with real-time variable status changes is powerful and can save significant development and manual testing time.

## Cost optimization

1. Route simpler tasks to cheaper models and reserve higher-cost models for ambiguous cases. Also, use keyword-based match first when possible, and only fall back to the model-based check when the keyword result is unclear. For example, the *check_confirmation_message* function can use a cheaper model and keyword-based matching.
2. Use caching where safe, reduce unnecessary model/tool calls, and enforce token budgets.
3. Monitor cost per successful resolution and optimize prompts, context size, and retry behaviour.